DexTrace

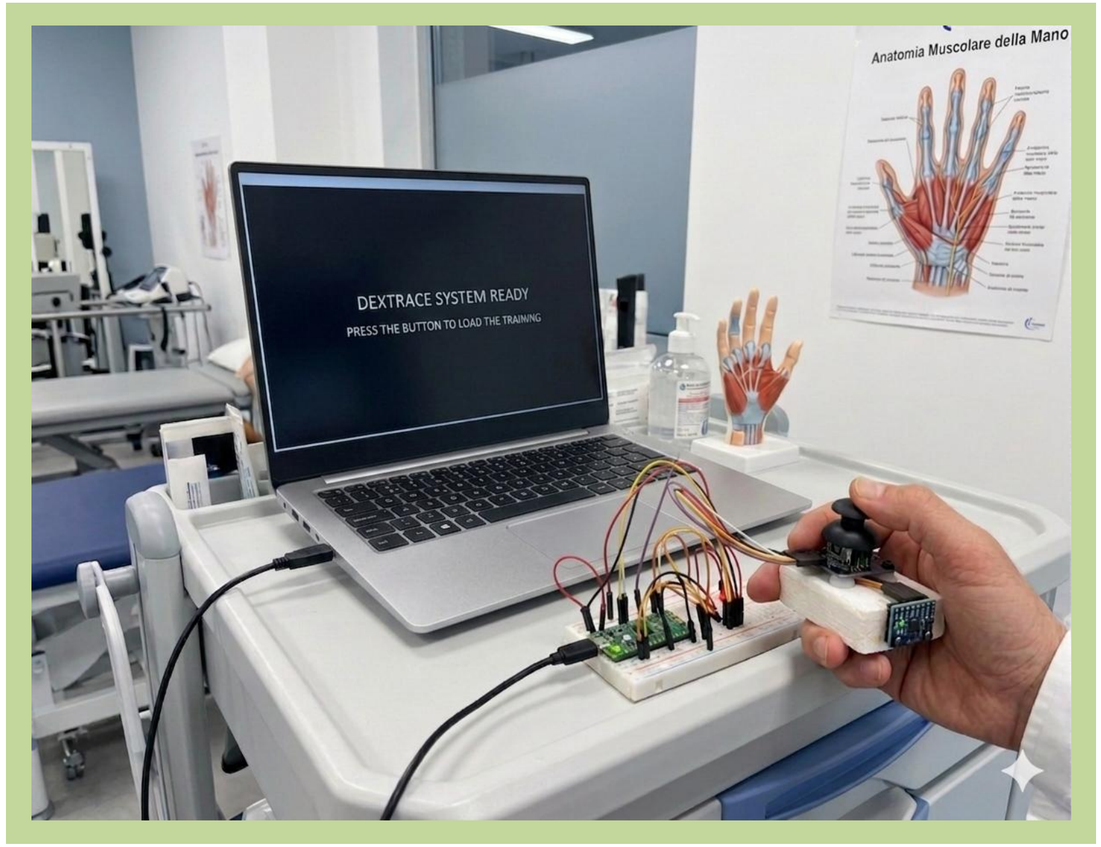
Rehabilitation for hand fine motor coordination and dexterity

The project

DexTrace is an integrated hardware-software system designed to **train** and **rehabilitate hand fine motor coordination** and **dexterity**.

By combining a **Raspberry Pi Pico** controller with a **Python-based GUI**, the system evaluates user performance across three progressive difficulty levels, monitoring **spatial precision** and **tremor intensity** in **real-time.**

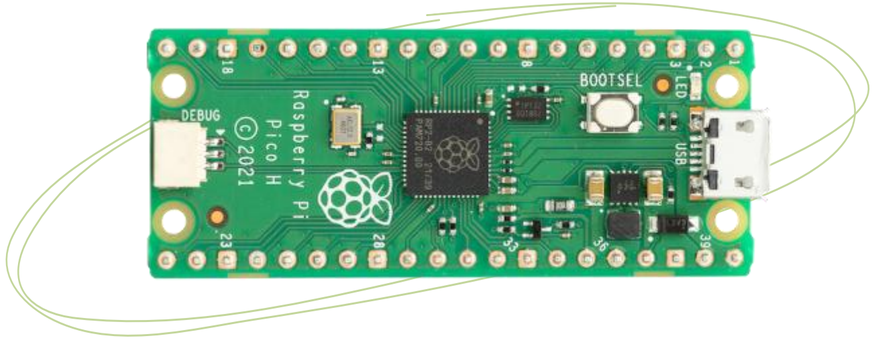
The key concept is to recover from injuries in a playful way.



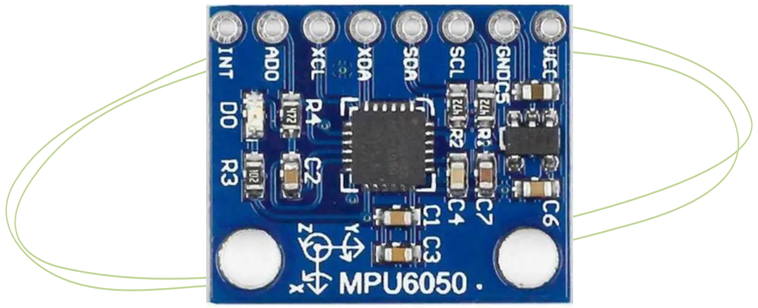
System Requirements

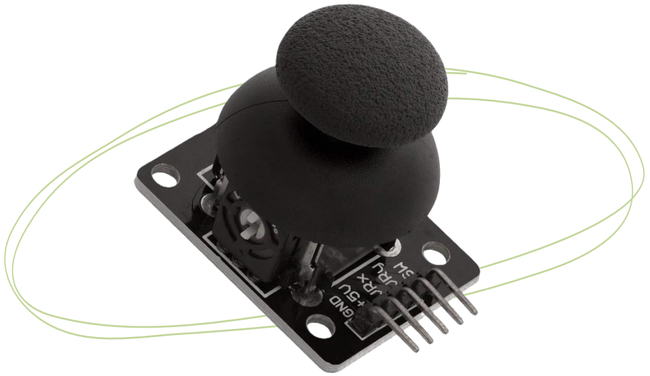
Hardware

Raspberry PI Pico

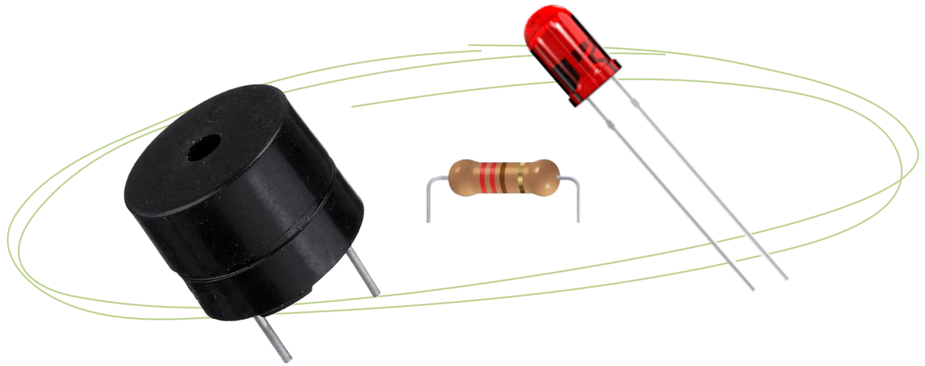


IMU sensor





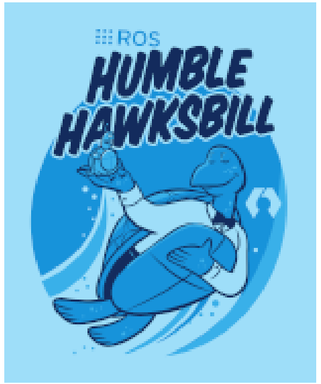
Analog Joystick (2-axis with integrated button)



Active buzzer and Led

System Requirements

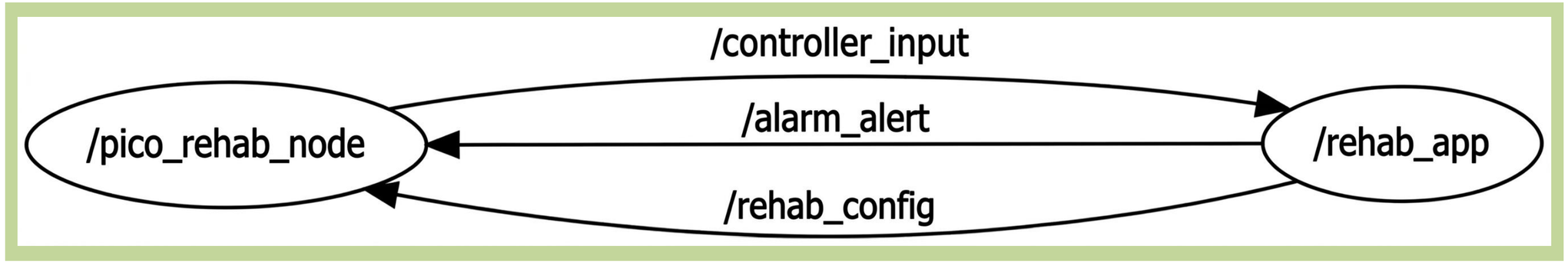
Software



Wiring Table

PICO PIN	Component PIN
38	Joystick GND, IMU GND
36	Joystick VCC, IMU VCC
31 - GP26	Joystick VRx
32 - GP27	Joystick VRy
27 - GP21	Joystick SW
7 - GP5	IMU SCL
6 - GP4	IMU SDA
20 - GP15	Buzzer
21 - GP16	Led

Communication ROS2 + uROS



Nodes:

- pico_rehab_node → Raspberry PI Pico
- rehab_app → PC

Topics:

- rehab_config → Level configuration
- controller_input → Joystick and IMU
- alarm_alert → Acoustic alarm

Messages

/rehab_config	/controller_input	/alarm_alert
geometry_msgs/msg/Twist	geometry_msgs/msg/Twist	std_msgs/msg/Bool
linear.x = Level gain linear.y = Eval threshold	linear.x = Joystick VRx*gain linear.y = Joystick VRy*gain angular.x = Tremor intensity angular.z = System state	data = target_alarm
Pico is a subscriber	Pico is a publisher	Pico is a subscriber

Standard message types with a custom field mapping (only for Twist messages) to optimize the bandwidth

Data Structures

```
struct SystemState {
    bool active;
    float current_tremor_intensity;
    DeviceConfig_t config;
};
```

This approach guarantee:
- **temporal coherence;**
- **producer-consumer decoupling.**

`SystemState` keeps the current state `active` or not of the system, the `current_tremor_intensity` from the IMU and the `config` received from the rehap_app node, defined as:

```
typedef struct {
    float gain;
    float tremor_threshold;
} DeviceConfig_t;
```

The configuration parameters are level-dependant. Its access is protected by a **Mutex**.

```
typedef struct {
    float x;
    float y;
    float tremor;
    bool active;
} SensorPacket_t;
```

`SensorPacket_t` is the current snapshot of the sensors ready to be sent. Each packet is filled in the **xSensorQueue**, with **xQueueOverwrite** (dim = 1) to keep only the last new fresh packet.

Task Scheduling & priorities

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Buzzer	≈160 ms	Event driven	4	1	Manages real-time acoustic feedback for target chase with minimum latency
Tremor	10 ms	100 Hz	4	1	Samples MPU6050 raw data via I2C and computes tremor intensity using variance analysis
Input	10 ms	100 Hz	3	1	Handles ADC joystick readings, dynamic gain scaling, and start-button software debouncing
ROS	40 ms	25 Hz	2	0	Executes the micro-ROS executor, manages serial transport, and implements the safety watchdog
Heartbeat	1000 ms	1 Hz	1	0	Provides visual system health status and session state via the onboard LED blinking frequency.

Task Buzzer

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Buzzer	≈160 ms	Event driven	4	1	Manages real-time acoustic feedback for target chase with minimum latency

Event: Asynchronous response to the /alarm_alert signal.

Real-Time Analysis: Although the sound is slow (80 ms cycles), its activation must be immediate. By increasing the priority to 4, it ensure that the buzzer can also preempt the tremor calculation if necessary, to alert the user.

Jitter Analysis: Variable latency of ROS2 communication but mitigated by the high priority and isolation on Core 1. It ensures a temporally coherent acoustic stimulus, essential to not confuse the patient's neuro-motor feedback during exercise.

Task Tremor

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Tremor	10 ms	100 Hz	4	1	Samples MPU6050 raw data via I2C and computes tremor intensity using variance analysis

Real-Time Analysis: This task must be deterministic. If the accelerometer sampling (via I2C) is delayed, the variance calculation over 20 samples (sliding window) becomes physically meaningless.

Global Resources: Writes to the global_state (Mutex-protected) and sends the data to SensorPacket_t.

Local Feedback: Immediate visual alert via LED if intensity exceeds threshold, no ROS communication jitter.

Task Input

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Input	10 ms	100 Hz	3	1	Handles ADC joystick readings, dynamic gain scaling, and start-button software debouncing

Real-Time Analysis: It manages the ADC (Analog-to-Digital Converter). There is a software filtering option to prevent electrical noise from the ADC from moving the cursor when the joystick is idle. It has the same T as the **Task Tremor** and based on the **Rate Monotonic Scheduling** they should have the same priorities, but the decision is to give more importance to the signal precision.

Debouncing: The button is handled in software with a vTaskDelay(50). It didn't use interrupts to avoid electrical glitches that could crash the Levels FSM (Finite State Machine).

Task ROS

TASK	T	F	PRIORITY	CORE	DESCRIPTION
ROS	40 ms	25 Hz	2	0	Executes the micro-ROS executor, manages serial transport, and implements the safety watchdog

Real-Time Analysis: This is the most demanding and unpredictable task. The micro-ROS stack is complex and manages serial buffers, so this task is assigned to Core 0, protecting hardware tasks (Core 1) from potential slowdowns caused by USB/Serial communication.

Watchdog: Implementation of a Fail-Safe design. If the PC stops sending data the Pico enters a safe state.

Task Heartbeat

TASK	T	F	PRIORITY	CORE	DESCRIPTION
Heartbeat	1000 ms	1 Hz	1	0	Provides visual system health status and session state via the onboard LED blinking frequency.

Real-Time Analysis: This is a "best-effort" task. It runs only when the CPU has nothing better to do. Changing the blink rate based on the active state is an excellent way to perform visual debugging without external tools.

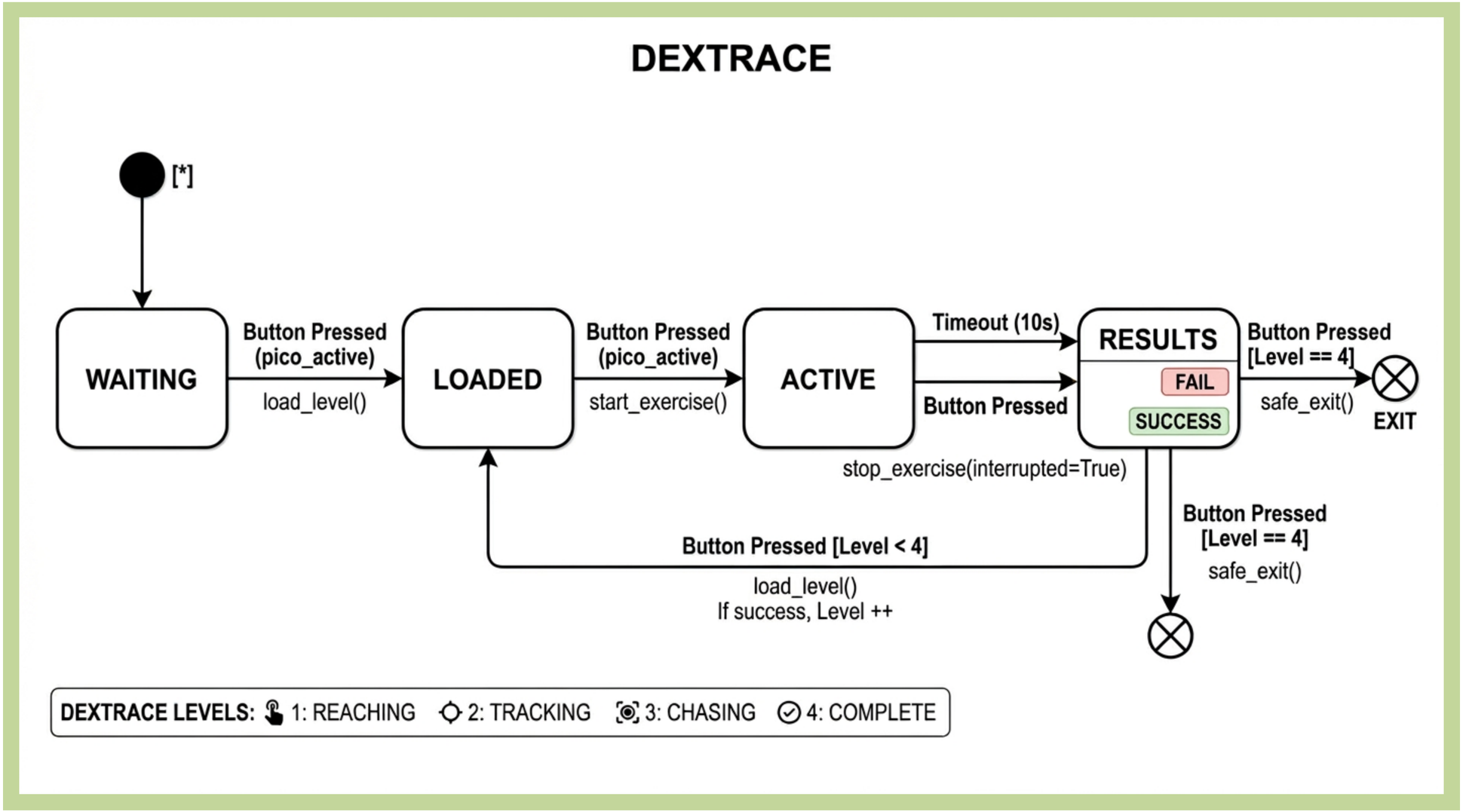
Clinical Evaluation Protocol

- Level 1 (*Reaching*): Path efficiency and target acquisition assessment;
- Level 2 (*Tracking*): Smooth circular target following;
- Level 3 (*Chasing*): Non-linear dynamics to simulate complex scenarios.

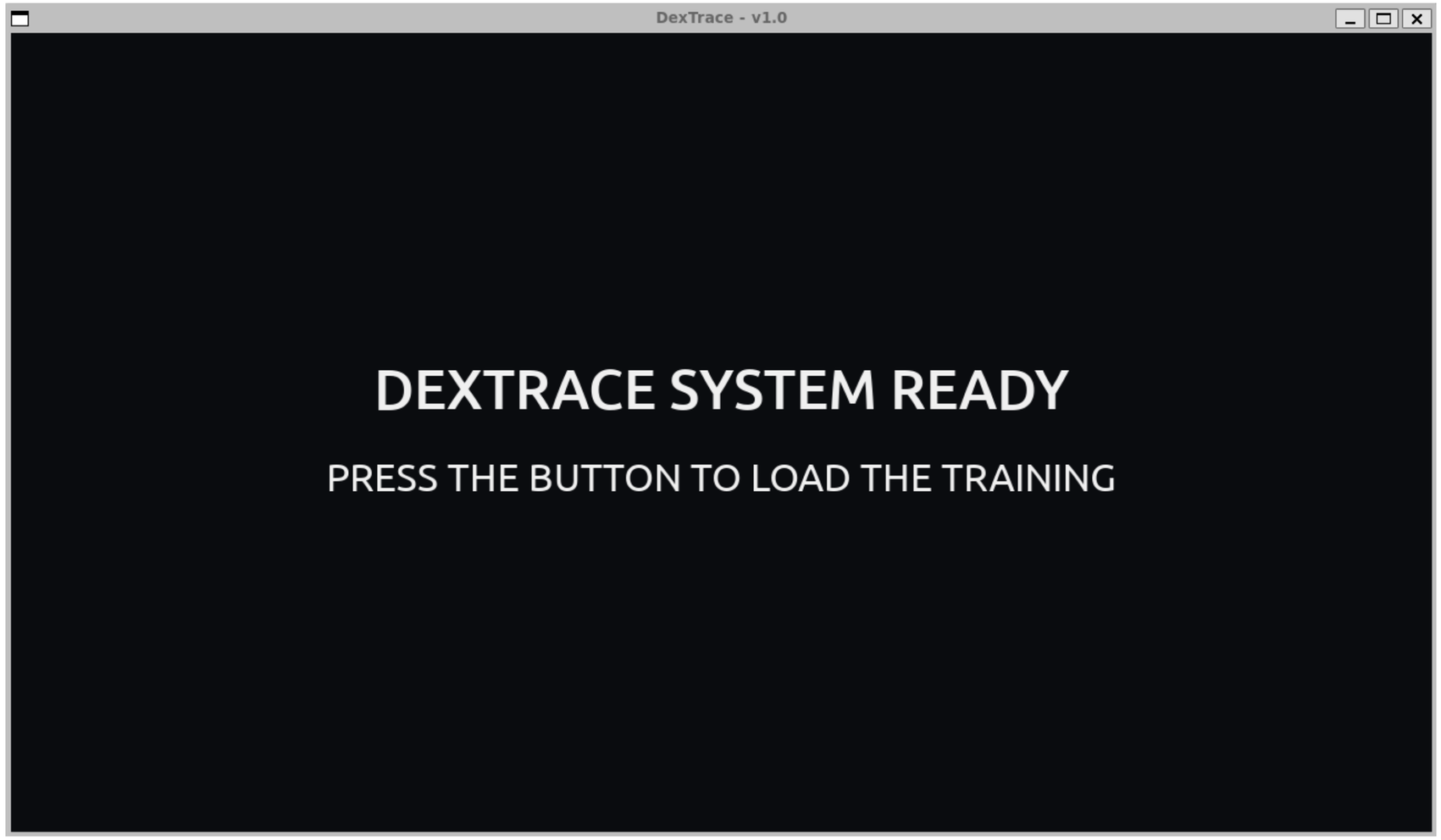
Dex-Metrics

Metric	Description	Formula	Level
Tremor intensity	Calculated based on the variance of 3-axis acceleration from the MPU6050	$Var(Mag_{accel})$	All
Dex-Efficiency	Ratio between real and optimal path + tremor penalty	$Path_{eff} + (Tremor_{avg} \cdot 0.5)$	1
Dex-Accuracy	RMSE weighted by tremor intensity	$\mathrm{RMSE} + (Tremor_{avg} \cdot 10)$	2 & 3

GUI - FSM

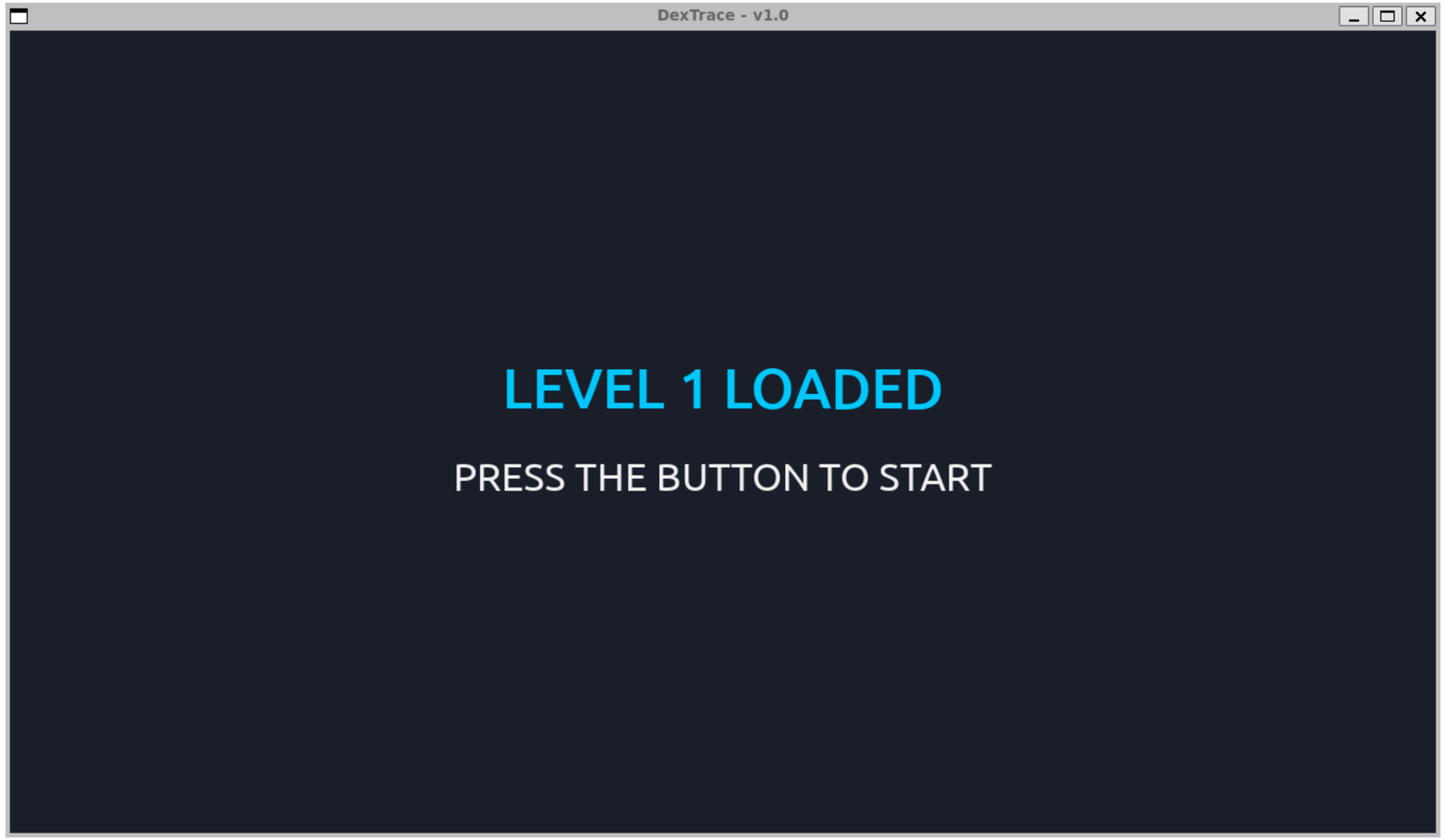

DEXTRACE
[*]
WAITING
Button Pressed
(pico_active)
load_level()
LOADED
Button Pressed
(pico_active)
start_exercise()
ACTIVE
Timeout (10s)
Button Pressed
RESULTS
FAIL
SUCCESS
Button Pressed
[Level == 4]
safe_exit()
EXIT
stop_exercise(interrupted=True)
Button Pressed [Level < 4]
load_level()
If success, Level ++
Button Pressed
[Level == 4]
safe_exit()
DEXTRACE LEVELS: 1: REACHING 2: TRACKING 3: CHASING 4: COMPLETE

Homepage

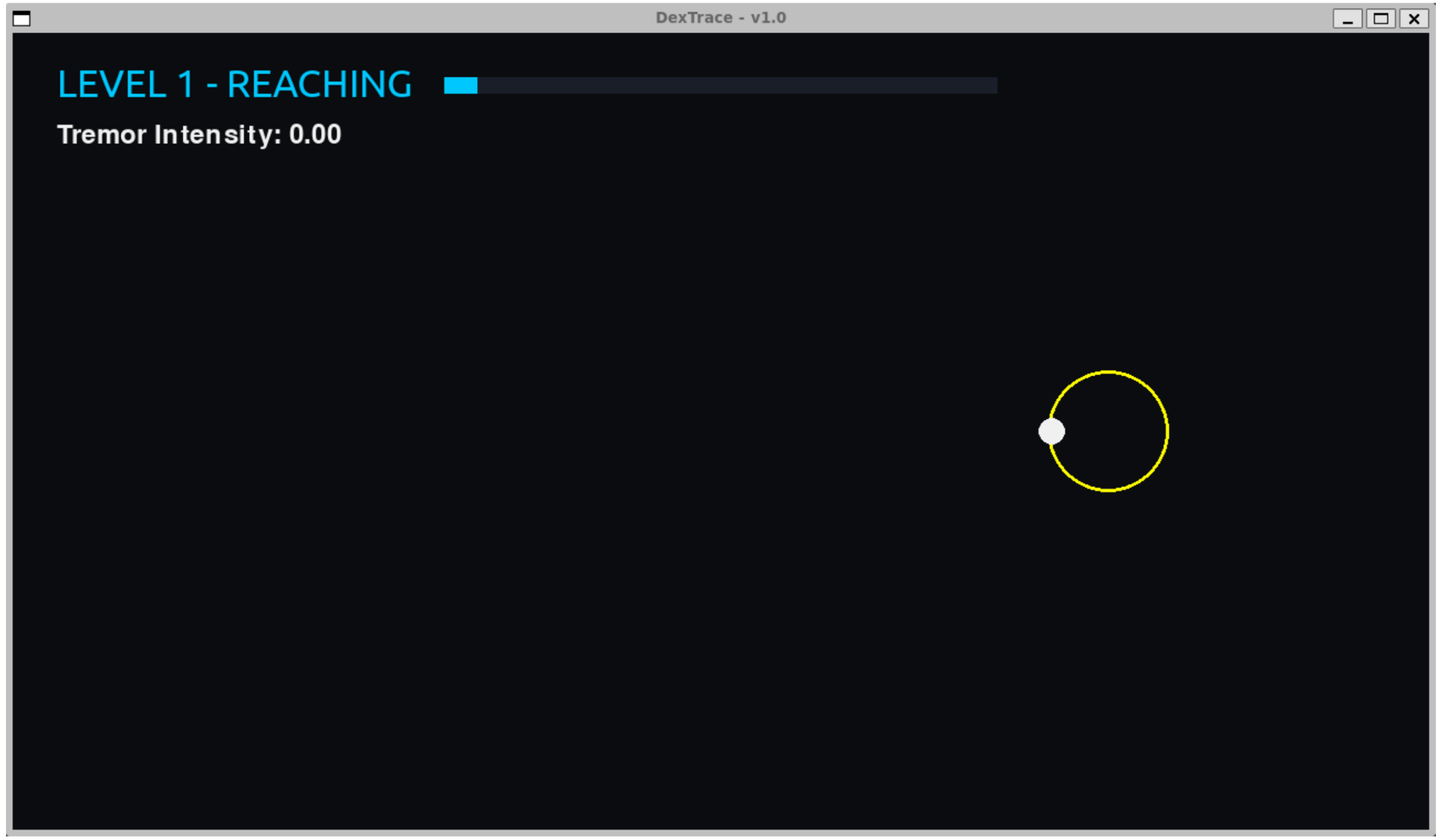


Library: pygame

Load Level

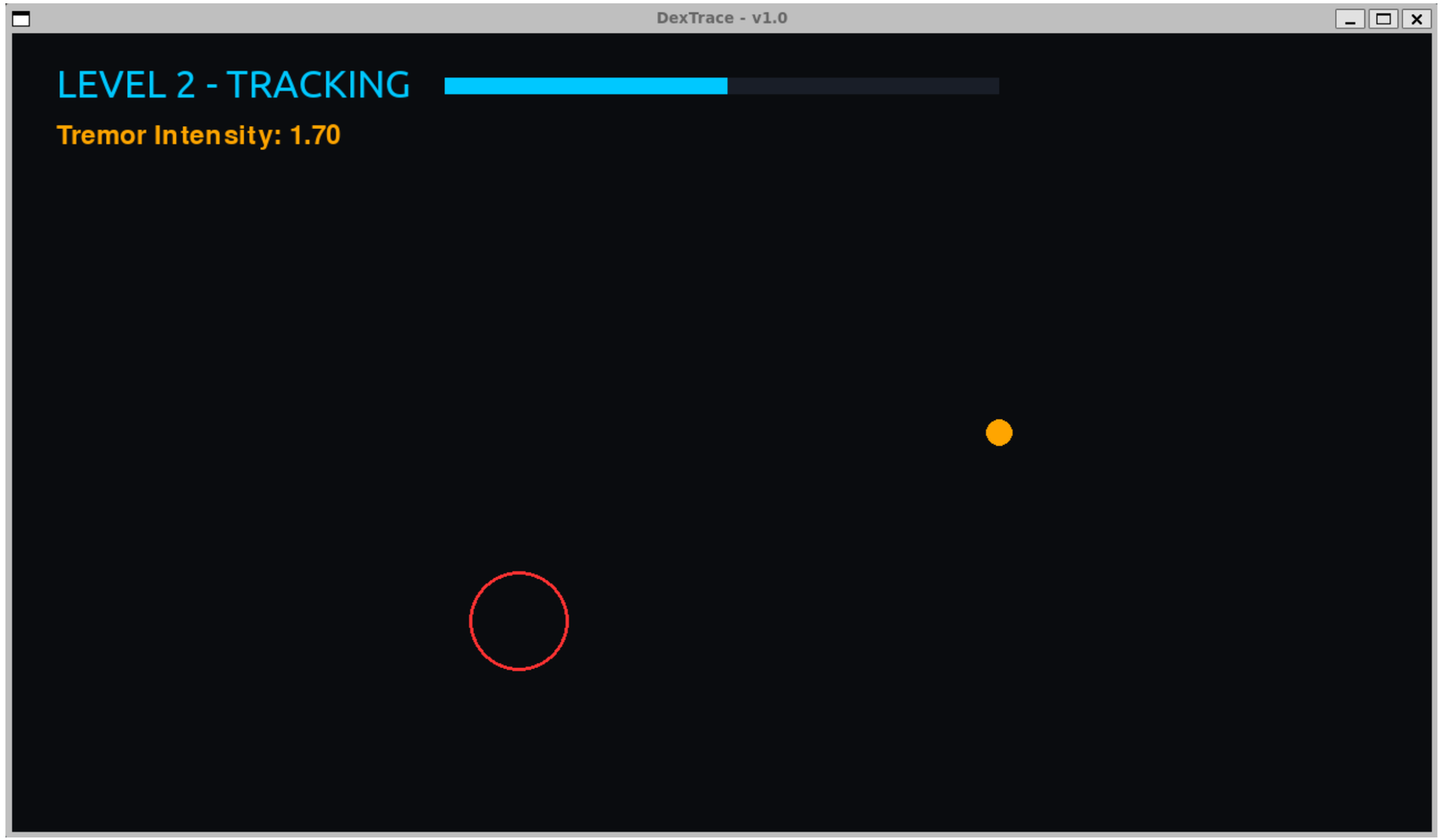


Level 1



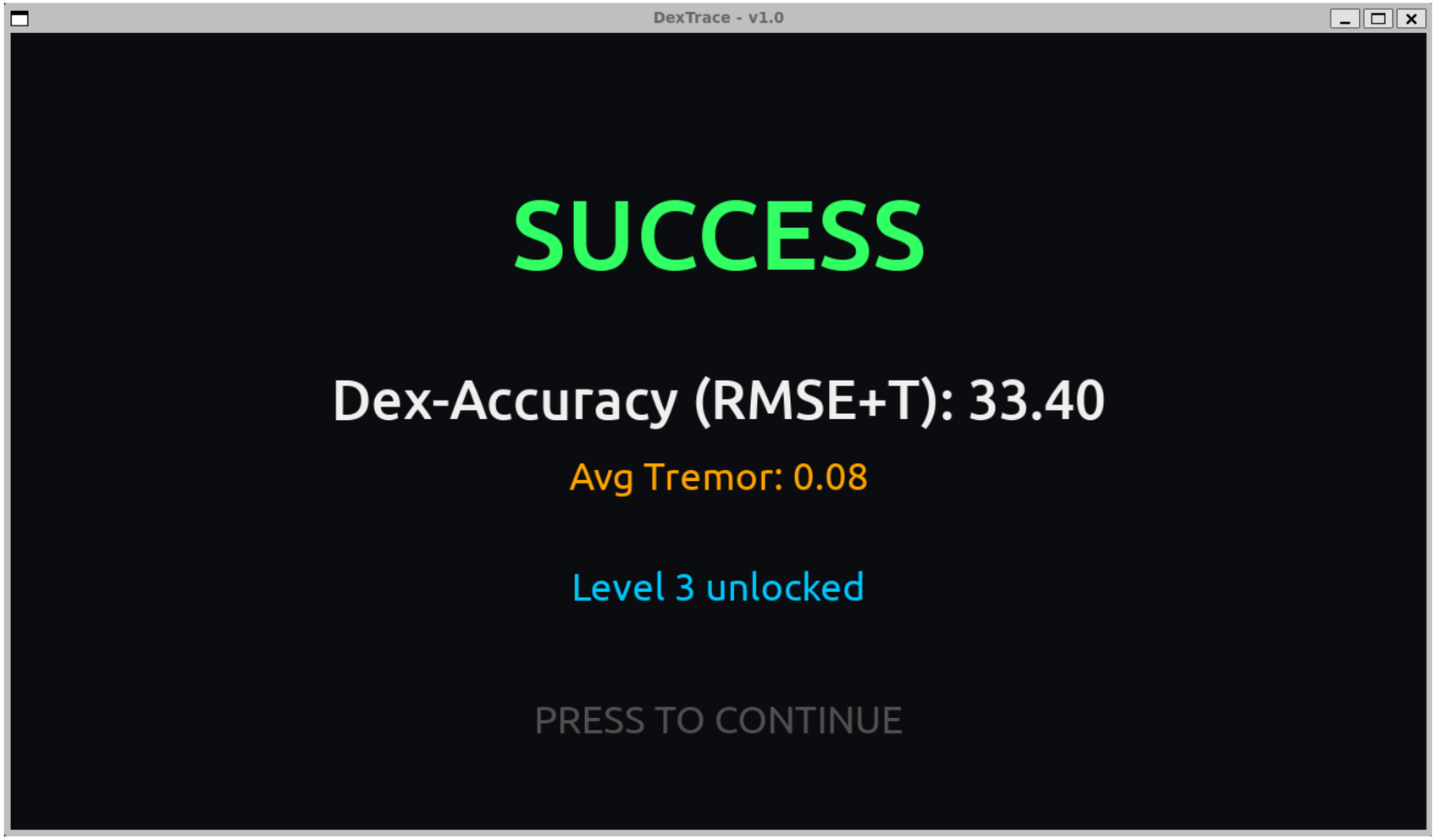
The target color is cursor-position dependant: red (not chased), yellow (on the border), green (chased)

Level 2

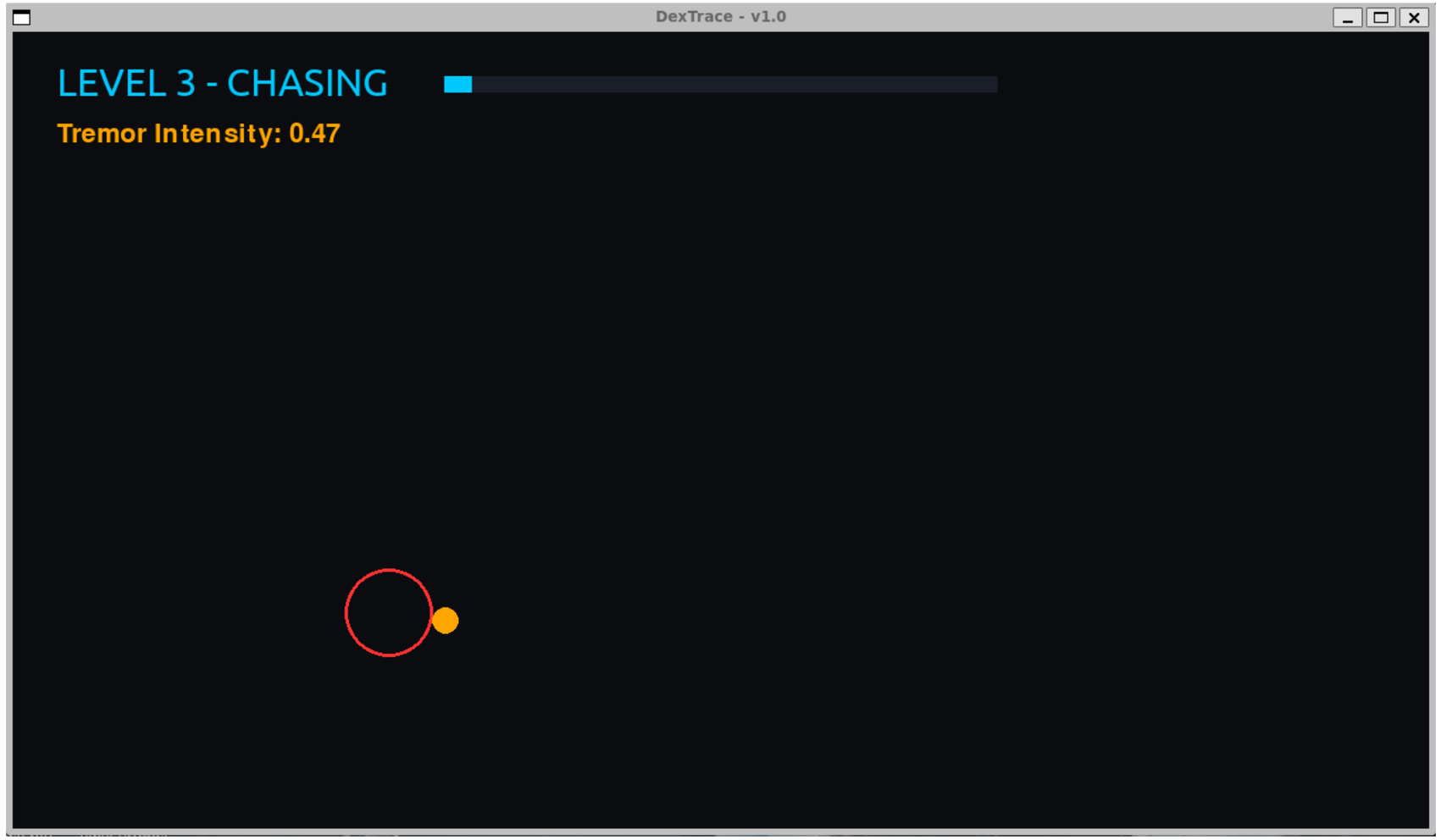


The cursor color became orange if the tremor intensity value (upper left) is greater than the threshold

Results


DexTrace - v1.0
SUCCESS
Dex-Accuracy (RMSE+T): 33.40
Avg Tremor: 0.08
Level 3 unlocked
PRESS TO CONTINUE

Level 3



Thanks for your attention

Developed by

Alex Veronese

Email:

302474@studenti.unimore.it

GitHub:

https://github.com/alexveronese